

To Flatten a Stone with Six Birds: Critical Pairs, Holonomy, and Confluence in Rewriting Systems

Ioannis Tsiokos 

v1: 17 March 2026 v2: 1 October 2026

Abstract

Confluence says that the order in which rewrite steps are applied does not matter in the end; flatness says that, locally, transport does not depend on the route taken. This paper makes the analogy precise at the local level and tests it computationally. The natural carrier is not the cycle structure of the reduction graph, and we show by small counterexamples that cycle-based notions of graph flatness neither imply nor are implied by confluence. Instead, we record each branching of two one-step rewrites out of a common state, the configuration tested by local confluence. A branching has trivial *elementary holonomy* when its two branches can be completed to a common descendant, so that its boundary closes up, and nontrivial holonomy when they cannot. With this definition, elementary flatness is exactly local confluence. Under termination, Newman’s lemma turns the local statement into a global one: confluence, elementary flatness, and uniqueness of normal forms coincide. These results hold for arbitrary abstract rewriting systems and are checked in Lean 4 without finiteness assumptions. In string and term rewriting, the branchings that can fail are the rule overlaps recorded by critical pairs; we work with the concrete critical-pair instances that occur in terms reachable from chosen start terms. An exhaustive enumeration of all 66,066 rewriting relations on one to four labeled states agrees with the theorems and yields smallest counterexamples showing that each hypothesis is needed. Curated string and left-linear term-rewriting examples, including an overlap below the root, show reachable critical-pair defects coinciding with the observed obstructions. Finally, two before-and-after examples read a completion step as holonomy elimination: the added rule leaves the branching in place but supplies the missing filler.

1 Introduction

A rewriting system is *confluent* when the choices made during reduction do not matter in the end: whenever a term can be rewritten in two ways, the two results can be rewritten further to a common term [1, 6, 3, 12]. Discrete geometry has a principle with the same flavor. A connection is *flat* when transport around every small, contractible loop returns what it started with, so that transport between two points is unchanged when the route is deformed with its endpoints fixed; the failure of flatness is measured by holonomy around such small loops [8, 2]. Flatness is thus a local form of route independence; around loops that cannot be contracted, a flat connection may still have nontrivial holonomy. The resemblance between “order does not matter” and “route does not matter” is easy to state and easy to overstate. This paper asks what the resemblance amounts to when it is made exact.

The answer depends on choosing the right object. A first guess is to look for flatness in the reduction graph itself, by asking whether the graph has cycles or how many independent undirected

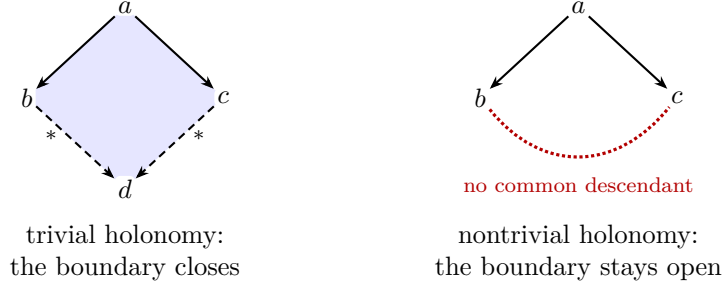


Figure 1: The local picture behind the paper. A peak $b \leftarrow a \rightarrow c$ is a pair of one-step rewrites out of a common state. If both branches reduce to a common state d (dashed arrows denote rewrite sequences), the peak and the joining paths bound a closed region that can be filled by a 2-cell. If no such d exists, there is nothing to fill. Local confluence is the statement that every peak is of the first kind.

cycles it contains. Section 3 shows that this fails in both directions: there are acyclic graphs that are not confluent and cyclic graphs that are. The reason is simple. Confluence compares pairs of rewrite sequences leaving the same state, and its local form, local confluence, compares pairs of one-step rewrites; by Newman’s lemma, the local form suffices for terminating systems. Such a pair of one-step rewrites, a *peak*, is not a cycle. It becomes the boundary of a closed region only once the two branches have been brought back together.

This suggests recording the peaks explicitly. Following the higher-dimensional view of rewriting [9, 5, 4, 10], we treat states as vertices, one-step rewrites as edges, and each peak as a candidate two-dimensional cell. When the two branches of a peak have a common descendant, the peak and its joining paths form a closed boundary and the cell can be filled; we then say that the peak has trivial *elementary holonomy*. When no common descendant exists, the boundary stays open and the holonomy is nontrivial (Figure 1). The resulting structure, which we call a reduction 2-complex, is the object on which the flatness reading of confluence becomes literal.

With these definitions the local correspondence is exact: a system is *elementarily flat*, meaning that every peak has trivial holonomy, if and only if it is locally confluent (Theorem 5.2). The proof is short because the definitions are designed to match. Its value lies less in its difficulty than in fixing a precise dictionary. That dictionary becomes more informative when combined with termination. By Newman’s lemma, a terminating system that is elementarily flat is confluent, and, for a terminating system, confluence, elementary flatness, and uniqueness of reachable normal forms are equivalent (Section 5). We prove these statements for arbitrary abstract rewriting systems and check them in Lean 4 (Appendix A).

Concrete rewriting formalisms add one further layer. In string and term rewriting, most peaks are harmless: two rewrites at disjoint positions commute, and a rewrite inside the substituted part of another redex can be absorbed. The only peaks that can fail to be joinable come from genuine overlaps of left-hand sides, which are recorded by *critical pairs* [7, 6, 1]. Critical pairs therefore generate the nontrivial part of the holonomy question (Section 6). In computations we work relative to chosen start terms and use the *reachable* critical-pair instances: concrete overlaps, possibly inside a larger context and under a substitution, that occur in terms reachable from the starts.

Contributions.

1. We show by explicit small counterexamples that the two natural cycle-based notions of flatness for reduction graphs are unrelated to confluence (Proposition 3.1), and we explain why: peaks are not cycles.

2. We define reduction 2-complexes and elementary holonomy as a fillability status of peaks, and prove that elementary flatness is equivalent to local confluence (Theorem 5.2). Under termination we obtain the global form: confluence, elementary flatness, and unique normal forms coincide (Corollary 5.7, Theorem 5.8). All results are mechanized in Lean 4 for arbitrary rewriting relations.
3. We place critical pairs in this picture as the generators of the nontrivial holonomy obligations in string and term rewriting, and distinguish them from peaks that are joinable for structural reasons (Section 6).
4. We test the picture computationally: an exhaustive enumeration of all rewriting relations on one to four labeled states, smallest counterexamples for each dropped hypothesis, and curated string and left-linear term-rewriting examples, including an overlap below the root (Section 7).
5. We read two explicit completion steps, one for strings and one for terms, as holonomy elimination: the branching survives, its filler appears, and confluence is restored (Section 8).

Organization. Section 2 fixes notation and places the paper within the Six Birds program. Section 3 disposes of the cycle-based reading. Section 4 introduces reduction 2-complexes and elementary holonomy, and Section 5 proves the main results. Section 6 treats critical pairs, Section 7 reports the computations, and Section 8 discusses completion. Section 9 separates what is proved from what is computed and what remains open.

2 Preliminaries and Scope

2.1 Abstract rewriting

An *abstract rewriting system* is a pair $(X, \rightarrow)$ consisting of a set X of states and a binary relation $\rightarrow \subseteq X \times X$, the one-step rewrite relation. No finiteness is assumed. We write $\rightarrow^*$ for the reflexive-transitive closure of $\rightarrow$, and say that b is *reachable* from a , or is a *descendant* of a , when $a \rightarrow^* b$. The following notions are standard [1, 6, 12].

Definition 2.1 (Basic notions). Let $(X, \rightarrow)$ be an abstract rewriting system.

1. States $b, c \in X$ are *joinable*, written $b \downarrow c$, if there is a $d \in X$ with $b \rightarrow^* d$ and $c \rightarrow^* d$.
2. A state n is a *normal form* if there is no m with $n \rightarrow m$.
3. The system is *locally confluent* if $a \rightarrow b$ and $a \rightarrow c$ imply $b \downarrow c$.
4. The system is *confluent* if $a \rightarrow^* b$ and $a \rightarrow^* c$ imply $b \downarrow c$.
5. The system is *normalizing* if every state reaches some normal form, and *terminating* if there is no infinite rewrite sequence $a_0 \rightarrow a_1 \rightarrow a_2 \rightarrow \dots$.
6. A state a has *unique reachable normal forms* if any two normal forms reachable from a are equal. This asserts that there is *at most one*; existence is a separate matter.

Termination implies normalization, but not conversely; and neither confluence nor uniqueness of normal forms implies either of them. When X is finite, the system is the same thing as a finite directed graph, possibly with self-loops, and termination amounts to the absence of directed cycles.

In string and term rewriting, X is a set of words or first-order terms and $\rightarrow$ is generated by a set of rules applied at any position and, for terms, under any substitution. These settings have additional structure, notably critical pairs, which we recall in Section 6.

2.2 Place in the Six Birds program

Remark 2.2 (Six Birds primitives). This paper uses the closure vocabulary introduced in [13], whose six primitives are operator rewrite (P1), constraints or feasibility gating (P2), route mismatch or holonomy (P3), staging (P4), packaging (P5), and accounting or audit (P6). The present paper is primarily about P3: its subject is local route mismatch in rewriting systems. P5 and P6 play supporting roles, in packaging rewriting data into reduction 2-complexes and in keeping every defect explicit in machine-readable records. The mathematics below does not depend on this vocabulary.

Within that program, the paper is a mathematical companion to [14], which treats route mismatch as a quantitative defect, and it draws on the finite-state and coarse-graining background of [15]. Neither is needed to read what follows.

2.3 Three kinds of evidence

The paper combines three kinds of evidence, and we keep them separate throughout.

1. *Proved results.* The statements in Section 5 are theorems about arbitrary abstract rewriting systems, with paper proofs and machine-checked Lean proofs. Proposition 3.1 is proved by explicit counterexamples.
2. *Cited classical results.* The critical-pair lemma of Knuth–Bendix and Huet, which connects critical pairs to local confluence in string and term rewriting, is used as stated in the literature. We neither re-prove nor mechanize it.
3. *Computations.* An exhaustive enumeration of small finite systems, a search for smallest counterexamples, and a set of curated string and term-rewriting examples. Statements resting on these computations concern only the systems examined. Where a computation would need an unbounded search, as with term rewriting from a start term, a verdict is reported only after the set of reachable terms has been explored completely.

The paper does not claim a new confluence criterion for term rewriting, nor a holonomy theory in the sense of a group-valued connection. Its contribution is a precise local dictionary, the global theorems that follow from it, and a computational test of both.

3 Why Graph Cycles Are the Wrong Test

For a finite abstract rewriting system the reduction graph *is* the system: its vertices are the states and its directed edges are the one-step rewrites. Every property in Definition 2.1, confluence included, is therefore a property of that directed graph, and an algorithm can decide it from the graph by computing reachability. The question of this section is narrower. Graph theory and discrete geometry offer ready-made notions of flatness built from *cycles*, and one may hope that one of them captures confluence. We show that the two most natural ones do not.

1. *Directed-cycle flatness* calls the reduction graph flat when it has no directed cycle.
2. *Undirected cycle rank* measures the dimension of the cycle space of the underlying simple undirected graph, i.e. the number of independent undirected cycles; rank zero means the graph is a forest.

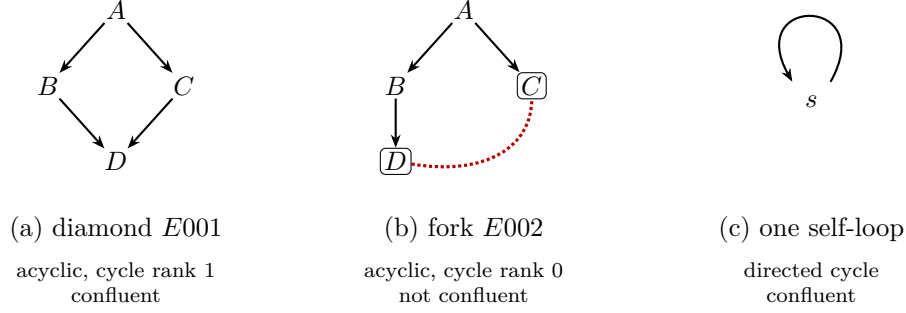


Figure 2: Cycle structure says nothing about confluence. (a) The diamond is confluent and has an undirected cycle. (b) The fork has no cycles of either kind but is not confluent: its two normal forms C and D (boxed) have no common descendant, as indicated by the dotted line. (c) A single state with a self-loop is confluent, since every state reachable from s is s itself, yet it has a directed cycle.

Table 1: Cycle-based proxies on the curated finite systems $E001$ – $E004$. All four graphs are acyclic, so directed-cycle flatness calls all of them flat, yet $E002$ and $E004$ are not confluent. Undirected cycle rank is positive on the confluent systems $E001$ and $E003$ and zero on the nonconfluent ones. The column of nonjoinable peaks is the local quantity that does track confluence (Section 5).

Example	Directed-cycle flat?	Undirected cycle rank	Nonjoinable peaks	Confluent?
$E001$ (diamond)	yes	1	0	yes
$E002$ (fork)	yes	0	1	no
$E003$ (delayed join)	yes	1	0	yes
$E004$ (double fork)	yes	0	2	no

Proposition 3.1 (Cycle-based proxies do not detect confluence). *Among finite abstract rewriting systems, neither directed-cycle flatness nor vanishing undirected cycle rank implies confluence, and neither is implied by confluence. Directed-cycle flatness fails to imply confluence even among terminating systems, where it holds by definition.*

Proof. All four claims are witnessed by the systems in Figure 2. The fork (b), with edges $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, has no directed cycle and its undirected graph is a tree, yet C and D are distinct normal forms reachable from A , so it is not confluent. The self-loop (c) has a directed cycle and is confluent. The diamond (a) has undirected cycle rank 1 and is confluent, so confluence does not force rank zero. Finally, every terminating finite system is acyclic and hence directed-cycle flat, while the fork is terminating and not confluent. $\square$

The exhaustive search of Section 7.2 finds these witnesses automatically: the smallest system on which directed-cycle flatness holds without confluence is the three-state fork $s_0 \rightarrow s_1$, $s_0 \rightarrow s_2$, and the smallest confluent system with a directed cycle is the one-state self-loop.

Why do cycle-based notions miss confluence? Because confluence compares pairs of rewrite sequences out of a common state, and locally, for instance in terminating systems where local confluence suffices, it is tested on *peaks*, pairs of one-step rewrites $b \leftarrow a \rightarrow c$. A divergence need not contain a cycle, and the existence of a cycle does not determine whether its branches are joinable. In the fork the peak at A cannot be joined, but nothing in the graph closes up around it; in the diamond the peak can be joined, and only then does it become part of a closed walk $A \rightarrow B \rightarrow D \leftarrow C \leftarrow A$. Cycles that do exist in a reduction graph may have nothing to do with peaks, as in the self-loop. A notion of flatness adapted to confluence must therefore first say *which*

pairs of paths are being compared. The reduction graph contains all the information needed to answer the question, but it does not single out the peaks as the objects on which the comparison takes place. The next section makes that comparison data explicit.

4 Reduction 2-Complexes and Elementary Holonomy

This section introduces the object on which the flatness reading of confluence is literal. It adds to the reduction graph a list of *branchings*, the pairs of one-step rewrites to be compared, and fills a branching with a 2-cell exactly when its two branches can be brought together.

4.1 Branchings and fillers

Definition 4.1 (Elementary peaks and fillers). Let $(X, \rightarrow)$ be an abstract rewriting system.

1. An *elementary peak* is a triple (a, b, c) with $a \rightarrow b$, $a \rightarrow c$, and $b \neq c$. We call a its *source* and b, c its *branches*. The triples (a, b, c) and (a, c, b) describe the same *branching*; when branchings are counted or cells attached, the two are identified and one representative is used.
2. A *filler* for the peak (a, b, c) is a state w , the *join witness*, together with rewrite sequences $b \rightarrow^* w$ and $c \rightarrow^* w$.

A filler turns the open shape $b \leftarrow a \rightarrow c$ into a closed walk

$$a \rightarrow b \rightarrow^* w \leftarrow^* c \leftarrow a,$$

which can serve as the boundary of a disk. Without a filler there is no closed boundary to attach anything to. This asymmetry is the whole point of the construction, and we build it into the definition.

Definition 4.2 (Reduction 2-complex). Let $(X, \rightarrow)$ be an abstract rewriting system and let G be a set of branchings, i.e. of elementary peaks up to exchange of the two branches, called the *generating branchings*. The *reduction 2-complex* of $(X, \rightarrow)$ with respect to G consists of

1. the states of X as 0-cells;
2. the one-step rewrites $a \rightarrow b$ as directed 1-cells;
3. for each branching in G that admits a filler, one 2-cell attached along the closed walk of a chosen filler.

Branchings in G that admit no filler are kept as *open branchings*: they are recorded, but no cell is attached to them.

For an abstract rewriting system, G is the set of all branchings. For string and term rewriting, Section 6 takes G to be the set of critical-pair instances, the branchings that can actually fail. Whether a branching admits a filler does not depend on which filler is chosen; the choice affects only which 2-cell is drawn. The computations of Section 7 fix a canonical filler by a deterministic rule (Appendix B) so that the recorded complexes are reproducible.

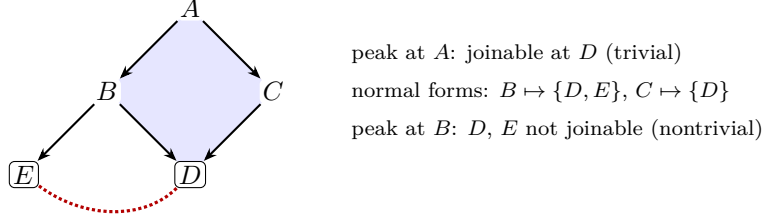


Figure 3: Joinability and normal-form agreement differ. The peak at A is filled (shaded) because B and C both reach D , even though the sets of normal forms reachable from B and from C differ. The obstruction to confluence sits at the peak $D \leftarrow B \rightarrow E$.

4.2 Elementary holonomy

Definition 4.3 (Elementary holonomy and elementary flatness). An elementary peak has *trivial elementary holonomy* if it admits a filler, and *nontrivial elementary holonomy* otherwise; the status is the same for (a, b, c) and (a, c, b) , so it is a property of the branching. A rewriting system is *elementarily flat* if every elementary peak has trivial elementary holonomy, that is, if the branches of every elementary peak are joinable. A reduction 2-complex records this status for the branchings in G : the filled ones carry 2-cells and the others remain open.

The term is chosen to evoke, not to import, the geometric notion. Elementary holonomy is a yes/no status attached to a single branching: either going down one branch and then on to w agrees with going down the other branch and then on to w , for some w , or no such w exists. It is not the holonomy of a connection valued in a group, and nothing below is a statement about fundamental groups, homotopy of rewrite paths, or higher coherence. Those richer questions are the subject of the homotopical theory of rewriting initiated by Squier. For a convergent (terminating and confluent) presentation of a monoid, Squier showed that a choice of confluence diagrams for the critical branchings yields a homotopy basis of the associated space of derivations, and the polygraphic setting extends this to higher dimensions [10, 11, 4, 5, 9]. Elementary holonomy records only the first, Boolean layer of that structure: whether a given 2-cell exists at all.

Remark 4.4 (Comparing normal forms is a different test). For terminating systems it is tempting to declare a peak defective when its two branches have different sets of reachable normal forms. This is not the same as nonjoinability, and it is not local. In the system of Figure 3, the peak at A is joinable at D , so its holonomy is trivial, yet B reaches the normal forms $\{D, E\}$ while C reaches only $\{D\}$. The discrepancy is caused by a different peak, the one at B , which is genuinely nonjoinable. In our computations, normal-form comparison is therefore recorded only as a secondary diagnostic, and only for terminating systems; elementary holonomy and all defect counts use joinability alone.

4.3 Representative examples

The same local question arises in each of the settings studied later: given two one-step rewrites from a common state, is there a filler? Table 2 lists the generating branchings and their holonomy in six representative examples. In the finite systems $E001$ and $E002$, the generating branchings are the peaks of Figure 2. In the string pair $E007$ – $E008$ and the term pair $TRS003$ – $TRS004$ they are critical-pair instances; these pairs are discussed in Sections 6 and 8.

Table 2: Generating branchings in six representative examples. A filled branching carries a 2-cell attached along the closed walk through its join witness; an open branching has nontrivial elementary holonomy and no attached cell.

Example	Domain	Branchings	Filled	Open	Join witness
$E001$ (diamond)	finite ARS	1	1	0	D
$E002$ (fork)	finite ARS	1	0	1	none
$E007$ (before)	string	1	0	1	none
$E008$ (after)	string	1	1	0	y
$TRS003$ (nested)	term	1	0	1	none
$TRS004$ (resolved)	term	1	1	0	$p(a)$

5 Elementary Flatness, Local Confluence, and Termination

Throughout this section $(X, \rightarrow)$ is an arbitrary abstract rewriting system. No finiteness is assumed, and every result has a Lean counterpart listed in Appendix A.

5.1 The local equivalence

Lemma 5.1 (Reflexive joinability). *Every state is joinable with itself.*

Proof. Take $d = x$ and use $x \rightarrow^* x$ twice. □

Theorem 5.2 (Elementary flatness is local confluence). *An abstract rewriting system is elementarily flat if and only if it is locally confluent.*

Proof. Suppose the system is locally confluent and let (a, b, c) be an elementary peak. Then $a \rightarrow b$ and $a \rightarrow c$, so $b \downarrow c$ by local confluence. Hence every elementary peak has a filler.

Conversely, suppose the system is elementarily flat and let $a \rightarrow b$ and $a \rightarrow c$. If $b = c$, then $b \downarrow c$ by Lemma 5.1. If $b \neq c$, then (a, b, c) is an elementary peak, which has a filler by assumption, so again $b \downarrow c$. □

Remark 5.3 (What the theorem says). Elementary flatness is local confluence restricted to peaks with distinct branches, so the theorem is close to a restatement; its only extra content is the trivial case $b = c$. We state it as a theorem because it is the exact form of the dictionary between the two vocabularies: “every elementary branching has trivial holonomy” and “every one-step divergence can be rejoined” are the same condition. The theorem says nothing about which peaks need to be checked in a structured formalism; that is the role of critical pairs (Section 6). Nor is it, by itself, a local-to-global statement. Termination supplies the global step.

5.2 From local to global under termination

The remaining results relate the local condition to confluence and to normal forms. The first is standard.

Proposition 5.4 (Confluence and normalization give a unique normal form). *If $(X, \rightarrow)$ is confluent and normalizing, then every $a \in X$ reaches exactly one normal form.*

Proof. Normalization gives a normal form n with $a \rightarrow^* n$. If also $a \rightarrow^* m$ with m a normal form, confluence gives a d with $n \rightarrow^* d$ and $m \rightarrow^* d$. A normal form has no descendants other than itself, so $d = n$ and $d = m$; hence $n = m$. □

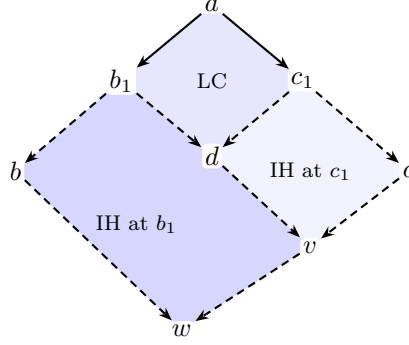


Figure 4: The tiling in the proof of Newman’s lemma. Local confluence (LC) fills the elementary peak at a ; the induction hypothesis (IH), available because b_1 and c_1 are one step below a , fills the two larger regions. Termination is what makes the induction well founded.

Uniqueness alone, the “at most one” half of this argument, uses only confluence. The next lemma is the familiar fact that termination yields normal forms. We use the formulation of termination as well-foundedness: every nonempty set of states contains an element with no one-step successor inside the set. This permits induction along $\rightarrow$, in which a property may be assumed for all one-step successors of a when proving it for a .

Lemma 5.5 (Termination implies normalization). *A terminating abstract rewriting system is normalizing.*

Proof. By well-founded induction along $\rightarrow$. If a is a normal form, it reaches itself. Otherwise $a \rightarrow b$ for some b ; by the induction hypothesis b reaches a normal form n , and so $a \rightarrow b \rightarrow^* n$. $\square$

Theorem 5.6 (Newman’s lemma). *A terminating, locally confluent abstract rewriting system is confluent.*

Proof. We prove by well-founded induction along $\rightarrow$ that every a satisfies: whenever $a \rightarrow^* b$ and $a \rightarrow^* c$, we have $b \downarrow c$. If either sequence is empty, say $a = b$, then c itself is a common descendant of b and c . Otherwise both sequences begin with a step, $a \rightarrow b_1 \rightarrow^* b$ and $a \rightarrow c_1 \rightarrow^* c$. By local confluence there is a d with $b_1 \rightarrow^* d$ and $c_1 \rightarrow^* d$. Since $a \rightarrow c_1$, the induction hypothesis applies at c_1 ; it joins c and d at some v . Since $a \rightarrow b_1$ and $b_1 \rightarrow^* d \rightarrow^* v$, the induction hypothesis applies at b_1 ; it joins b and v at some w . Then $b \rightarrow^* w$ and $c \rightarrow^* v \rightarrow^* w$. $\square$

Figure 4 displays the proof as a tiling: one elementary 2-cell and two cells supplied by induction. This is the precise sense in which, for terminating systems, global flatness is assembled from elementary flatness. Combining Theorem 5.6 with Theorem 5.2, and noting that confluence trivially implies local confluence, we obtain the global form of the dictionary.

Corollary 5.7 (Confluence is elementary flatness under termination). *A terminating abstract rewriting system is confluent if and only if it is elementarily flat.*

The relation between confluence and normal forms needs less than termination.

Theorem 5.8 (Confluence and unique normal forms). *Let $(X, \rightarrow)$ be normalizing. Then $(X, \rightarrow)$ is confluent if and only if every state has unique reachable normal forms.*

Proof. If the system is confluent, two normal forms reachable from the same state are joinable and hence equal, as in the proof of Proposition 5.4. Conversely, suppose every state has unique reachable

normal forms, and let $a \rightarrow^* b$ and $a \rightarrow^* c$. By normalization, $b \rightarrow^* n$ and $c \rightarrow^* m$ for normal forms n and m . Both are reachable from a , so $n = m$, and this common normal form joins b and c . $\square$

Corollary 5.9 (Existence and uniqueness of normal forms). *In a terminating, elementarily flat abstract rewriting system, every state reaches exactly one normal form.*

Proof. By Corollary 5.7 the system is confluent, by Lemma 5.5 it is normalizing, and Proposition 5.4 applies. $\square$

For terminating systems, then, four conditions coincide: elementary flatness, local confluence, confluence, and uniqueness of reachable normal forms. Each hypothesis used along the way is needed. Section 7.2 exhibits smallest finite witnesses: a four-state system that is locally confluent but not confluent, for which termination fails (Figure 6); a one-state system that is confluent but not normalizing; and a three-state system in which every state has unique reachable normal forms but which is not confluent, for which normalization fails.

Remark 5.10 (Mechanization). All results in this section are proved in Lean 4 for an arbitrary relation on an arbitrary type, with no finiteness assumption and with termination stated as well-foundedness of the reduction relation in the direction of rewriting. The proofs use no axioms beyond Lean’s standard ones (propositional extensionality, quotient soundness, and choice), and several use none. Appendix A gives the correspondence.

6 Critical Pairs as Generators of Holonomy

In string and term rewriting, the one-step relation is generated by finitely many rules applied anywhere, so a single word or term typically has many peaks. Elementary flatness asks for all of them to be joinable. This section recalls why only a small, structured part of them can fail, namely the overlaps recorded by critical pairs, and explains how we use critical pairs relative to chosen start terms.

6.1 Three kinds of peaks

Consider a term t with two redexes, one for a rule $l_1 \rightarrow r_1$ at position p and one for a rule $l_2 \rightarrow r_2$ at position q . There are three possibilities [6, 1, 12], illustrated in Figure 5.

1. *Disjoint redexes.* Neither position lies below the other. The two rewrites act on separate parts of t and commute: each result can be rewritten by the other rule to the same term. For example, with rules $a \rightarrow b$ and $c \rightarrow d$, the word ac has the peak $bc \leftarrow ac \rightarrow ad$, joined at bd . No two left-hand sides overlap here.
2. *Variable overlap.* One redex lies inside the part of the other that is matched by a variable. The peak is again joinable: on the side where the outer rule has fired, the inner rewrite is applied to every copy of that variable’s instance on the right-hand side; on the other side, it is applied to any remaining copies on the left-hand side, after which the outer rule fires. For left-linear rules there are no remaining copies. For example, with rules $f(x) \rightarrow g(x)$ and $a \rightarrow b$, the term $f(a)$ has the peak $g(a) \leftarrow f(a) \rightarrow f(b)$, joined at $g(b)$.
3. *Critical overlap.* One redex lies at a non-variable position of the other’s left-hand side, so the two left-hand sides genuinely share symbols. Joinability is not automatic. Every such peak is an instance, inside some context and under some substitution, of a *critical pair*: a pair of

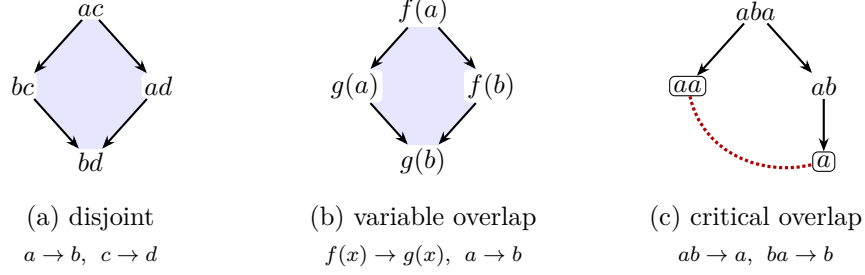


Figure 5: The three kinds of peaks. (a) Rewrites at disjoint positions commute. (b) A rewrite inside the instance of a variable is absorbed after the outer rule fires. Both are always joinable. (c) Overlapping left-hand sides: in aba , the redexes ab and ba share the middle letter. This is the string example $E006$; its branches lead to the distinct normal forms aa and a (boxed), so the branching stays open.

terms obtained by unifying one left-hand side with a non-variable subterm of another, using a most general unifier. A rule may overlap with a renamed copy of itself at a proper subterm; the overlap of a rule with itself at the root is excluded because it produces two identical terms.

String rewriting is the special case of term rewriting with unary function symbols, and the same classification applies to words. Two left-hand sides overlap critically when a suffix of one is a prefix of the other, or when one occurs inside the other.

The first two cases are always joinable, so a nonjoinable peak must come from a critical overlap. Conversely, if every critical pair is joinable, then so is every instance of it in any context, because rewriting is closed under contexts and substitutions. This is the content of the classical critical-pair lemma.

Theorem 6.1 (Critical-pair lemma [7, 6]). *A term rewriting system is locally confluent if and only if all of its critical pairs are joinable. The same holds for string rewriting systems.*

We use this theorem as stated in the literature (see also [1, Chapter 6]); it is neither re-proved nor mechanized here. Combined with Theorem 5.2 and Corollary 5.7, it gives the holonomy form of the Knuth–Bendix criterion.

Corollary 6.2. *A term or string rewriting system is elementarily flat if and only if all of its critical pairs are joinable. If the system is terminating, it is confluent if and only if all of its critical pairs are joinable.*

In the language of Section 4, critical pairs generate the nontrivial holonomy obligations: they are the branchings whose fillers must be checked, while disjoint and variable-overlap peaks are filled automatically. They are not the whole set of peaks. Even a joinable critical pair is a genuine generating branching, whose 2-cell happens to be filled.

6.2 Critical pairs relative to start terms

Our computations study a rewriting system from designated start terms. Let S be the set of starts and X_S the set of terms reachable from S . Every descendant of a term in X_S lies in X_S , so the rewriting relation restricts to X_S , and two terms of X_S are joinable in the restricted system exactly when they are joinable in the full one.

Definition 6.3 (Reachable critical-pair instance). *A reachable critical-pair instance is a peak $u \leftarrow t \rightarrow v$ with $t \in X_S$ and $u \neq v$ in which the two redexes form a critical overlap. It is recorded by its source t , its unordered pair of branches $\{u, v\}$, and the rules and positions involved.*

Table 3: Critical pairs computed from the rules (“system”), reachable critical-pair instances, peaks in the explored reduction graph, and nonjoinable reachable instances. In every example the explored graph is complete, and the reachable instances coincide with the peaks.

Example	Domain	System CPs	Reachable instances	Graph peaks	Nonjoinable instances	Confluent?
<i>E005</i> (sort)	string	1	1	1	0	yes
<i>E006</i> (overlap)	string	2	1	1	1	no
<i>E007</i> (before)	string	1	1	1	1	no
<i>E008</i> (after)	string	1	1	1	0	yes
<i>TRS002</i> (root)	term	1	1	1	1	no
<i>TRS003</i> (nested)	term	1	1	1	1	no
<i>TRS004</i> (resolved)	term	1	1	1	0	yes

Two features of this definition matter. First, the instance is concrete. The source of a critical pair as computed from the rules is a most general overlap, such as ab for the rules $ab \rightarrow x$ and $ab \rightarrow y$, or $f(g(x))$ for $f(g(x)) \rightarrow p(x)$ and $g(y) \rightarrow q(y)$. The terms actually reached from the starts contain instances of it: the word zab , or the ground term $f(g(a))$. Asking whether the general source itself is reachable misses these. Second, the definition depends on S . A critical pair may have no instance in X_S , in which case it plays no role in the restricted system.

The classification of Section 6.1 applies peak by peak, and the joins of disjoint and variable-overlap peaks stay inside X_S . Hence the restricted system on X_S is locally confluent if and only if every reachable critical-pair instance is joinable. This is the start-relative form of Theorem 6.1 that the computations test. The equivalence holds whether or not X_S is finite. Our computational certification of it, however, requires complete exploration of X_S : the absence of a join witness in an incomplete bounded search does not certify nonjoinability (Section 7.5).

6.3 Curated examples

Table 3 compares, for the curated string and term examples, the number of critical pairs computed from the rules, the number of reachable critical-pair instances, and the number of peaks in the explored reduction graph. Since disjoint and variable-overlap peaks are joinable, the general relation is only that every *nonjoinable* peak is a critical-pair instance. In the curated examples something stronger happens: every peak of the explored graph is a critical-pair instance and conversely. This is a property of these examples, which happen to contain no disjoint or variable-overlap peaks, and not a general fact.

The string example *E006* (Figure 5c) has rules $ab \rightarrow a$ and $ba \rightarrow b$, and the rules produce two critical pairs, from the overlaps aba and bab . From the start word aba the reachable words are aba , aa , ab , and a . Only the first critical pair has an instance there, and it is the single peak of the explored graph. Its branches aa and $ab \rightarrow a$ end in different normal forms, so it is nonjoinable and the system is not confluent from aba . The second critical pair is irrelevant from this start.

Observation 6.4 (Curated string examples). In each of the string examples *E005*–*E008*, explored completely from its start word, the reachable critical-pair instances are exactly the peaks of the explored reduction graph, the nonjoinable instances are exactly the nonjoinable peaks, and the system is confluent from its start word exactly when there are none.

The supporting computations are in Tables 3 and 6. Each example is small enough to check by hand. In *E005*, the sorting rules $ba \rightarrow ab$, $ca \rightarrow ac$, $cb \rightarrow bc$ give, from cba , a single overlap of cb

and ba ; both branches bca and cab reach abc . In $E006$, as above, the single reachable instance is nonjoinable. The pair $E007$ – $E008$ is treated in Section 8.

The term examples test the same picture with first-order terms. In $TRS002$, the rules $f(a, y) \rightarrow g(y)$ and $f(x, b) \rightarrow h(x)$ overlap at the root of $f(a, b)$, giving the branches $g(b)$ and $h(a)$, which are distinct normal forms. In $TRS003$, the rules

$$f(g(x)) \rightarrow p(x), \quad g(a) \rightarrow b$$

overlap below the root: the redex $g(a)$ sits at the argument position of $f(g(x))$, a non-variable position. The reachable instance has source $f(g(a))$ and branches $p(a)$ and $f(b)$, again distinct normal forms. In $TRS004$ the rule $f(b) \rightarrow p(a)$ is added; the same instance remains, but now $f(b) \rightarrow p(a)$ fills it.

Observation 6.5 (Curated left-linear term examples). In each of the term examples $TRS002$ – $TRS004$, explored completely from its start term, the reachable critical-pair instances are exactly the peaks of the explored reduction graph, and the system is confluent from its start term exactly when every instance is joinable. In $TRS003$ and $TRS004$ the overlap lies below the root.

These observations are small. Their purpose is to confirm on concrete inputs that the implementation finds the right objects: instances in context rather than general overlap sources, overlaps below the root as well as at it, and a verdict tied to joinability.

7 Computational Evidence

The computations serve two purposes. For finite abstract rewriting systems, where the results of Section 5 are theorems, they check the implementation against the theorems and locate the smallest systems showing that each hypothesis is needed. For string and term rewriting, they test the start-relative critical-pair picture of Section 6 on curated examples. The implementation, the generated records, and the regression tests are described in Appendices B and C.

7.1 Exhaustive enumeration of small systems

We enumerate every rewriting relation on n labeled states for $1 \leq n \leq 4$, self-loops included. There are 2^{n^2} such relations, giving

$$2^1 + 2^4 + 2^9 + 2^{16} = 66,066$$

systems in all. A finite system is terminating exactly when its graph has no directed cycle, so the terminating systems are the labeled directed acyclic graphs; there are 1, 3, 25, and 543 of them for $n = 1, \dots, 4$, in agreement with the known count of labeled acyclic digraphs, for 572 in total. Of these, 1, 3, 22, and 401 are confluent.

For each terminating system we compute elementary flatness from the nonjoinable peaks, and independently compute local confluence, confluence, and uniqueness of reachable normal forms. Theorem 5.2, Corollary 5.7, and Theorem 5.8 predict that these four properties agree on every terminating system. They do: all four discrepancy counts in Table 4 are zero. Since the statements are proved, the value of this computation is as a test of the implementation, which the string and term computations reuse. As a further check, an independent implementation based on Boolean-matrix transitive closure recomputes termination, local confluence, confluence, and unique normal forms for all 66,066 systems, cyclic ones included, and agrees with the main implementation on every one.

Table 4: Exhaustive enumeration of rewriting relations on $n = 1, \dots, 4$ labeled states. On every terminating system, elementary flatness (EF), local confluence (LC), confluence (Conf), and unique reachable normal forms (UNF) agree, as Section 5 predicts; the last four columns count disagreements.

n	Systems	Terminating	Confluent terminating	EF $\neq$ LC	EF & $\neg$ Conf	Conf & $\neg$ UNF	UNF & $\neg$ Conf
1	2	1	1	0	0	0	0
2	16	3	3	0	0	0	0
3	512	25	22	0	0	0	0
4	65 536	543	401	0	0	0	0
total	66 066	572	427	0	0	0	0

Table 5: Smallest counterexamples to six plausible strengthenings, found by searching all relations on one to four labeled states (fewest states first, then fewest edges). “Normal-form peak test” declares a peak sound when its branches reach the same set of normal forms.

Refuted statement	Witness (edges)	Why it fails
Local confluence implies confluence without termination	$s_0 \rightarrow s_1, s_1 \rightarrow s_0, s_0 \rightarrow s_3, s_1 \rightarrow s_2$	locally confluent and normalizing, but s_0 reaches normal forms $s_2 \neq s_3$
Confluence implies normalization	$s_0 \rightarrow s_0$	confluent, but s_0 reaches no normal form
Unique reachable normal forms imply confluence without normalization	$s_0 \rightarrow s_1, s_0 \rightarrow s_2, s_1 \rightarrow s_1$	s_1 reaches no normal form, so every state has at most one; s_1 and s_2 are not joinable
Normal-form peak test implies elementary flatness without termination	$s_0 \rightarrow s_0, s_1 \rightarrow s_1, s_2 \rightarrow s_0, s_2 \rightarrow s_1$	both branches at s_2 reach no normal form, yet s_0 and s_1 are not joinable
No directed cycle implies confluence	$s_0 \rightarrow s_1, s_0 \rightarrow s_2$	terminating fork with two normal forms
Confluence implies no directed cycle	$s_0 \rightarrow s_0$	confluent self-loop

7.2 Smallest counterexamples

The same enumeration, now over all 66,066 systems rather than only the terminating ones, searches for systems that refute plausible strengthenings of the results. For each statement it reports a smallest witness, first by number of states and then by number of edges. All six statements in Table 5 are refuted. Together they show that the hypotheses of Section 5 cannot be dropped and that the shortcuts of Section 3 fail.

The first witness is worth drawing (Figure 6). It is locally confluent, and in fact every state reaches a normal form, but the cycle between s_0 and s_1 lets the system drift toward either of two distinct normal forms. It is the classical example showing that Newman’s lemma needs termination [6, 1].

7.3 Curated string systems

The four string examples were chosen to exercise the start-relative critical-pair picture: a confluent sorting system ($E005$), a nonconfluent overlap with an irrelevant second critical pair ($E006$), and a before/after completion pair ($E007, E008$). Each was explored from its start word until no new



Figure 6: A smallest locally confluent system that is not confluent. Both peaks are joinable: $s_3 \leftarrow s_0 \rightarrow s_1$ is joined at s_3 via $s_1 \rightarrow s_0 \rightarrow s_3$, and $s_2 \leftarrow s_1 \rightarrow s_0$ is joined at s_2 via $s_0 \rightarrow s_1 \rightarrow s_2$. Yet s_0 reaches the distinct normal forms s_2 and s_3 . The system is not terminating, so Newman’s lemma does not apply.

Table 6: Curated string examples, each explored completely from its start word: size of the explored reduction graph, reachable critical-pair instances, nonjoinable instances, normal forms, and confluence from the start word.

Example	Rules	States	Edges	Reachable instances	Non-joinable	Normal forms	Confluent?
E005	$ba \rightarrow ab, ca \rightarrow ac, cb \rightarrow bc$	6	6	1	0	abc	yes
E006	$ab \rightarrow a, ba \rightarrow b$	4	3	1	1	a, aa	no
E007	$ab \rightarrow x, ab \rightarrow y$	3	2	1	1	x, y	no
E008	$E007 + x \rightarrow y$	3	3	1	0	y	yes

words appeared, so the explored graph is the complete set X_S of reachable words. Table 6 reports the results. In every case the number of nonjoinable reachable critical-pair instances is zero exactly when the system is confluent from its start word.

7.4 Curated left-linear term systems

The four term examples are a small Peano-addition system with no critical pairs (*TRS001*), a root overlap (*TRS002*), and the nested-overlap pair *TRS003–TRS004*, whose overlap lies below the root. All rules are left-linear, which the critical-pair generator requires. All four were explored completely. The pattern of Table 7 matches the string case: confluence from the start term holds exactly when every reachable critical-pair instance is joinable.

7.5 When a computation certifies a verdict

Exploring a string or term rewriting system from a start term may not terminate, so the explorer works under explicit bounds on depth, number of states, and term size. Absence of a common descendant within the bounds does not prove that none exists. A truncated exploration of the diamond $a \rightarrow b, a \rightarrow c, b \rightarrow d, c \rightarrow d$ that stops after one step, for instance, sees b and c but not d . The implementation therefore separates two kinds of output. Diagnostics computed on an explored graph, such as its peaks and their joinability within the graph, are always available but are labeled as properties of the explored graph and are marked as certified for the reachable system only when the exploration was complete. The critical-pair, holonomy, and completion analyses on which the tables of this paper rest refuse to run on an incomplete exploration and report the bounds as inconclusive instead; where they also compute critical pairs from the rules, they require that computation to be complete as well. Completeness here refers to the reachable set X_S , not to all words or terms. All curated examples above meet this requirement.

The inputs are also validated before analysis. For term rewriting, a rule may not have a variable as its left-hand side or introduce variables on its right-hand side that do not occur on its left, function symbols must be used with consistent arities, and the critical-pair generator accepts only left-linear rules. One-step rewriting itself supports non-left-linear matching.

Table 7: Curated left-linear term examples, each explored completely from its start term. Columns as in Table 6. The start terms are $add(s(z), s(z))$ for $TRS001$, $f(a, b)$ for $TRS002$, and $f(g(a))$ for $TRS003$ and $TRS004$.

Example	States	Edges	Reachable instances	Non-joinable	Normal forms	Confluent?
$TRS001$ (Peano addition)	3	2	0	0	$s(s(z))$	yes
$TRS002$ (root overlap)	3	2	1	1	$g(b), h(a)$	no
$TRS003$ (nested overlap)	3	2	1	1	$f(b), p(a)$	no
$TRS004$ ($TRS003 + f(b) \rightarrow p(a)$)	3	3	1	0	$p(a)$	yes

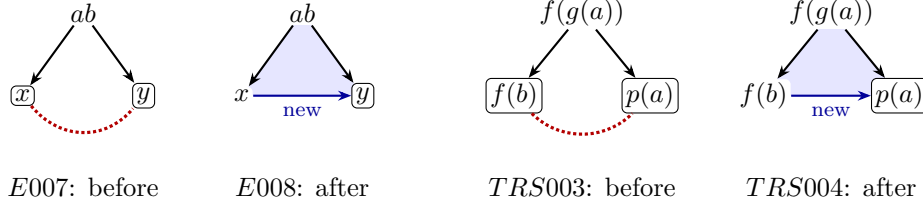


Figure 7: Completion as holonomy elimination. Left: the string rules $ab \rightarrow x$ and $ab \rightarrow y$ produce an open branching with distinct normal forms x and y ; adding $x \rightarrow y$ fills it with join witness y . Right: the term rules $f(g(a)) \rightarrow p(a)$ and $g(a) \rightarrow b$ produce an open branching at $f(g(a))$; adding $f(b) \rightarrow p(a)$ fills it with join witness $p(a)$. In both cases the source and the two branches are unchanged.

8 Completion as Holonomy Elimination

Knuth–Bendix completion repairs a nonconfluent system by orienting nonjoinable critical pairs into new rules, and, in its general form, also simplifies or removes existing rules [7, 3, 1]. We consider only the simplest kind of step, the addition of one rule. In the two rule-addition steps studied here, the overlap that produced the branching is still present after the new rule is added, so the branching remains; what changes is that it acquires a filler. We illustrate this with one string example and one term example, in each case adding a single rule by hand.

8.1 The two examples

The string system $E007$ has the rules $ab \rightarrow x$ and $ab \rightarrow y$, whose left-hand sides coincide. From the start word ab there is one branching, $x \leftarrow ab \rightarrow y$, and two normal forms, so the branching has nontrivial holonomy and the system is not confluent from ab . The system $E008$ adds the rule $x \rightarrow y$. The branching is still there, with the same source and branches, but now $x \rightarrow y$ provides a filler with join witness y . Its holonomy is trivial, and y is the unique normal form.

The term system $TRS003$ has the rules $f(g(a)) \rightarrow p(a)$ and $g(a) \rightarrow b$, which overlap below the root. From the start term $f(g(a))$ there is one branching, $p(a) \leftarrow f(g(a)) \rightarrow f(b)$, whose branches are distinct normal forms. The system $TRS004$ adds the rule $f(b) \rightarrow p(a)$, which provides a filler with join witness $p(a)$.

Observation 8.1 (Completion as holonomy elimination). In the string pair $E007$ – $E008$ and the term pair $TRS003$ – $TRS004$, explored completely from their start terms, the added rule

1. leaves the reachable critical-pair instance in place, with the same source and branches;
2. changes its elementary holonomy from nontrivial to trivial;

Table 8: Before and after the added rule. The number of branchings does not change; the number of nonjoinable ones drops to zero, and confluence from the start term is restored.

Domain	Before → after	Added rule	Branchings		Nonjoinable		Confluent?	
			before	after	before	after	before	after
string	$E007 \rightarrow E008$	$x \rightarrow y$	1	1	1	0	no	yes
term	$TRS003 \rightarrow TRS004$	$f(b) \rightarrow p(a)$	1	1	1	0	no	yes

3. reduces the number of nonjoinable reachable instances from one to zero; and
4. makes the system confluent from its start term.

In both pairs the “after” system consists of exactly the “before” rules plus the one added rule, with the same start term, so that the comparison is between a system and a one-rule extension of it. For the string pair this is checked automatically as part of the completion computation; for the term pair it can be read off the two rule sets.

8.2 What is eliminated

The examples separate two quantities that are easily conflated: the number of generating branchings and the number of nontrivial ones. Completion reduces the second while leaving the first unchanged. In the reduction 2-complex, the effect of the new rule is to add a 1-cell along which a 2-cell can now be attached; the branching that was open becomes the boundary of a filled cell.

Remark 8.2 (Scope). The two examples are hand-built single steps, chosen so that one added rule suffices. They illustrate a reading of completion; they are not a geometric formulation of the completion procedure. In general, an added rule can create new critical pairs, completion may fail or fail to terminate, and orienting an equation requires a reduction ordering [7, 1]. None of this is addressed here.

9 What Is Proved, What Is Computed, What Remains Open

Table 9 summarizes the status of each claim. This section comments on the three groups.

9.1 Proved

For abstract rewriting systems the picture is complete. Elementary flatness is local confluence (Theorem 5.2). Under termination, Newman’s lemma makes it equivalent to confluence (Corollary 5.7); under the weaker hypothesis of normalization, confluence is equivalent to uniqueness of reachable normal forms (Theorem 5.8); and a terminating, elementarily flat system has exactly one normal form reachable from each state (Corollary 5.9). These are classical facts of rewriting theory restated in the vocabulary of holonomy, and all of them are checked in Lean for arbitrary, possibly infinite, systems. The failure of cycle-based graph flatness (Proposition 3.1) is proved by explicit counterexamples.

9.2 Cited and computed

For string and term rewriting, the link between critical pairs and local confluence is the classical critical-pair lemma, which we cite (Theorem 6.1). Together with the results above it gives

Table 9: Status of the main claims. “Proved” means proved in the paper and, where indicated, in Lean. “Cited” means a classical result used as stated in the literature. “Computed” means verified on the specific systems examined. “Open” means not established here.

Claim	Evidence	Status
Elementary flatness $\iff$ local confluence (Theorem 5.2)	paper proof; Lean	proved
Confluence and normalization give a unique normal form (Proposition 5.4)	paper proof; Lean	proved
Newman’s lemma; under termination, confluence $\iff$ elementary flatness (Theorem 5.6, Corollary 5.7)	paper proof; Lean	proved
Under normalization, confluence $\iff$ unique reachable normal forms (Theorem 5.8)	paper proof; Lean	proved
Cycle-based graph flatness does not detect confluence (Proposition 3.1)	explicit counterexamples	proved
Critical-pair lemma and its holonomy form (Theorem 6.1, Corollary 6.2)	literature [7, 6]	cited
Theorems agree with all terminating systems on 1–4 states; each hypothesis needed (Tables 4 and 5)	exhaustive enumeration	computed
Reachable critical-pair instances match the observed obstructions in the curated string and term examples (Observations 6.4 and 6.5)	complete exploration of curated examples	computed
A single added rule fills the open branching in two curated pairs (Observation 8.1)	complete exploration of curated examples	computed
Mechanized critical-pair lemma for left-linear term rewriting in 2-complex form	—	open
Group-valued or connection-based holonomy theory of confluence	—	open

Corollary 6.2: a system is elementarily flat if and only if its critical pairs are joinable, and, when terminating, confluent under the same condition.

The computations add evidence of a different kind. On finite systems with one to four states they agree with the theorems on every case and produce smallest counterexamples for each dropped hypothesis. On the curated string and term examples they show that the start-relative analysis, based on concrete critical-pair instances in reachable terms, finds exactly the observed obstructions, including an overlap below the root, and that a single added rule can turn an open branching into a filled one. These are statements about the examples examined, and we do not extrapolate them.

9.3 Open

Three directions remain open.

1. *A mechanized critical-pair lemma.* The Lean development covers abstract rewriting only. A formal proof of Theorem 6.1 and its start-relative form for left-linear term rewriting, phrased in terms of reduction 2-complexes, requires the full case analysis of Section 6.1 together with stability of rewriting under contexts and substitutions. It is not attempted here.
2. *A genuinely geometric formulation.* Elementary holonomy is a Boolean status. A theory in which holonomy takes values in a group, or in which confluence becomes the flatness of an independently defined connection on a reduction complex, would need definitions that this

paper does not supply. The homotopical approach to rewriting [10, 4, 5, 9] is the natural setting for such a development.

3. *Completion in general.* Section 8 reads two single completion steps as holonomy elimination. Whether the full completion procedure, with its orderings, new critical pairs, and possible failure, admits a useful description in these terms is open.

10 Conclusion

The analogy between confluence and flatness is exact once one compares the right things. Cycles in the reduction graph are the wrong things: they neither detect nor are detected by confluence. Confluence compares pairs of rewrite sequences from a common state; local confluence compares the two branches of each peak, and under termination the two conditions are equivalent. A peak becomes a closed boundary, and so something that can be filled, only when its branches have a common descendant. Recording peaks as generating branchings, and filling them when possible, turns the reduction graph into a reduction 2-complex on which elementary holonomy is defined. Elementary flatness is then exactly local confluence, and under termination it is exactly confluence. The proof of Newman’s lemma reads as a tiling of a large region by elementary cells, and these statements are checked in Lean for arbitrary rewriting systems.

In string and term rewriting, critical pairs mark the branchings that can fail; disjoint and variable-overlap peaks are filled automatically. Working relative to chosen start terms, with concrete critical-pair instances and with verdicts certified only for completely explored reachable sets, the computations find exactly the expected obstructions on the curated examples. In two completion examples, the added rule leaves the branching in place and supplies its filler.

The central message can be stated without exaggeration. Local order-independence of reduction, the joinability of every peak, is exactly elementary flatness, and for terminating systems it is equivalent to full order-independence, that is, to confluence. On the reduction 2-complex, the local generators of nontrivial holonomy are the nonjoinable peaks, and in structured rewriting they are instances of critical pairs.

A Lean Formalization

The results of Section 5 are formalized in Lean 4 (version 4.28.0) in the package `ConfluenceFlat`, which has no external dependencies. The development covers abstract rewriting only; the string and term-rewriting computations are not formalized.

A.1 Setting

The basic structure is a rewriting system on an arbitrary type, given by a single relation `step`. There is no finiteness assumption, and every definition and theorem below quantifies over such systems. A separate structure for finite systems extends it with a finite list of states containing every state that occurs in a step; it is provided for representing concrete examples and is not used by any theorem. Reachability is the inductively defined reflexive–transitive closure of `step`. Joinability, normal forms, local confluence, elementary peaks (with distinct branches), elementary flatness, confluence, normalization, and unique reachable normal forms are defined exactly as in Definitions 2.1, 4.1 and 4.3. Termination is well-foundedness of the relation “ b is a one-step successor of a ”, so that well-founded induction proceeds along rewrite steps. Uniqueness of reachable normal forms means “at most one”; existence and uniqueness together is a separate predicate.

A.2 Correspondence

Table 10 lists the Lean theorem corresponding to each result. All names are in the namespace `ConfluenceFlat.FiniteARS`, a name retained for continuity although the theorems are not restricted to finite systems.

Table 10: Results of Section 5 and their Lean counterparts.

Result	Lean theorem
Lemma 5.1	<code>joinable_refl</code>
Theorem 5.2	<code>elementaryFlat_iff_localConfluent</code>
Proposition 5.4	<code>existsUniqueNormalFormFrom_of_confluent_normalizing</code>
Lemma 5.5	<code>normalizing_of_terminating</code>
Theorem 5.6	<code>confluent_of_terminating_localConfluent</code>
Corollary 5.7	<code>confluent_iff_elementaryFlat_of_terminating</code>
Theorem 5.8	<code>confluent_iff_uniqueNormalForms_of_normalizing</code>
Corollary 5.9	<code>existsUniqueNormalFormFrom_of_terminating_elementaryFlat</code>

Supporting lemmas include `reachable_eq_of_normalForm` (a normal form reaches only itself), `eq_of_joinable_normalForms`, and `uniqueNormalFormFrom_of_confluent`. The two directions of Theorem 5.2 are `elementaryFlat_of_localConfluent` and `localConfluent_of_elementaryFlat`.

A.3 Axioms

There are no `sorry` placeholders and no custom axioms. According to Lean’s `#print axioms`, the formal versions of Proposition 5.4, Theorem 5.6, and Theorem 5.8, as well as the supporting lemma that confluence gives unique reachable normal forms, depend on no axioms at all. The formal versions of Theorem 5.2, Lemma 5.5, Corollary 5.7, and Corollary 5.9 depend only on Lean’s standard axioms `propext`, `Classical.choice`, and `Quot.sound`, which enter through classical case distinctions, such as whether two states are equal or whether a state has a successor.

B Computational Protocol

The computations are implemented in Python; the code, data, and generated records are available at <https://github.com/ioannist/six-birds-confluence>. This appendix summarizes the procedures behind Sections 3, 7 and 8.

B.1 Finite abstract rewriting systems

The exhaustive enumeration ranges over all subsets of $\{s_0, \dots, s_{n-1}\}^2$ for $1 \leq n \leq 4$, self-loops included. Termination is decided by a topological sort. For each terminating system, every elementary peak is classified as joinable or not by computing descendant sets, and local confluence, confluence, and uniqueness of reachable normal forms are computed separately; the four discrepancy counts of Table 4 compare these. The counterexample search of Table 5 evaluates the same predicates, together with normalization and the cycle-based proxies, on all 66,066 systems and keeps, for each refuted statement, the witness with fewest states, then fewest edges, then smallest identifier in enumeration order.

An independent check recomputes termination, local confluence, confluence, and uniqueness of reachable normal forms for all 66,066 systems using Warshall’s transitive-closure algorithm on

Boolean adjacency matrices, and compares the results with the main implementation, which uses breadth-first search. The two agree on every system, cyclic ones included, and on every terminating system the three properties local confluence, confluence, and unique normal forms coincide.

B.2 String and term rewriting

String and term systems are explored from their start terms by breadth-first search under explicit bounds on depth, number of states, and word length or term size. A start term that already exceeds the bounds is rejected rather than silently admitted. The empty word is a valid state, so length-reducing string rules may delete a word entirely.

Critical pairs from the rules. For strings, critical pairs are computed from suffix–prefix overlaps and inclusions of left-hand sides. For terms, a left-hand side is unified with each non-variable subterm of a renamed copy of another left-hand side, including the same rule at a proper subterm; overlaps of a rule with itself at the root are discarded. Unification uses a full occurs check, so that equations such as $f(x, y) = f(g(y), g(x))$ are correctly reported as having no unifier. Substitutions are applied simultaneously: a variable is replaced once, and the replacement is not substituted into again. Critical pairs that differ only by a renaming of variables are identified. The term-rewriting critical-pair generator accepts only left-linear rules; inputs with a variable left-hand side, with right-hand-side variables absent from the left-hand side, or with inconsistent function arities are rejected.

Reachable instances. Independently of the critical pairs computed from the rules, every explored term is scanned for pairs of redexes forming a critical overlap: overlapping occurrences for strings, and for terms, two redexes one of which lies at a non-variable position of the other’s left-hand side. Each such pair, recorded with its rules and positions, is a reachable critical-pair instance in the sense of Definition 6.3. Disjoint redexes and redexes inside variable instances are not recorded as instances. The comparison with graph peaks of Table 3 uses the signature consisting of the source and the unordered pair of branches.

Certification. An exploration is complete when it has reached every term reachable from the starts and found no further successors within the bounds. Graph-level diagnostics are computed for any exploration, scoped to the explored graph and flagged as certified for the reachable system only when the exploration is complete. The critical-pair, reachable-instance, holonomy, and completion analyses reject incomplete explorations, and incomplete computations of critical pairs from the rules, with an explicit message that the bounds are inconclusive (Section 7.5).

B.3 Holonomy records and fillers

A branching is recorded as filled exactly when its branches are joinable; comparison of reachable normal-form sets is stored separately as a diagnostic and is computed only when the explored system is terminating (Remark 4.4). For filled branchings, the join witness is chosen to minimize the total length of the two joining paths, with ties broken by the lexicographically smallest witness, and each joining path is a shortest path found by breadth-first search with successors visited in sorted order. Filled branchings are exported together with the closed boundary walk through the witness; open branchings are exported without one.

B.4 Completion pairs

For the string pair *E007–E008*, the completion computation checks that the “after” system has exactly the rules of the “before” system plus the declared new rule, and the same start word, before comparing the two. The term pair *TRS003–TRS004* is compared directly from its two explored systems.

B.5 Regression

A regression harness runs the unit test suite (126 tests, including dedicated tests for the edge cases mentioned above), regenerates the curated finite, string, term, completion, reduction 2-complex, and diagram records and the results index, and runs a reduced exhaustive enumeration over $1 \leq n \leq 3$ states. It fails on a failing test, on a discrepancy found by the reduced enumeration, or on any step that raises an error or exits unsuccessfully. The full enumeration over $1 \leq n \leq 4$ states, the counterexample search, the independent Warshall check, the Lean build and axiom report, and the summary records are produced separately.

C Artifact Guide

Every number in the tables of this paper is read from a machine-readable record in the repository. This appendix lists where those records live.

C.1 Records by family

- Finite abstract rewriting systems: the curated examples are in `data/examples/`; the exhaustive enumeration, the counterexample search, and the cycle-proxy comparison are in `results/audits/`; peak and filler records are in `results/peaks/` and `results/plaquettes/`.
- String and term rewriting: explored systems are in `results/string_rewrite/` and `results/term_rewrite/`, and critical-pair analyses, including the lists of reachable critical-pair instances, are in `results/critical_pairs/` and `results/term_rewrite/`.
- Reduction 2-complexes: records of 0-cells, 1-cells, and generating branchings, with filled or open status, join witnesses, and closed boundary walks, are in `results/two_complex/`; Graphviz diagram sources are in `results/diagrams/`.
- Completion pairs: the string pair is recorded as a case study in `results/case_studies/`; the term pair is recorded through its two explored systems in `results/term_rewrite/` and its branchings in `results/two_complex/`.
- Summary records: the claim-status matrix and the final experiment summary are in `results/freeze/`; `results/index.json` and `results/summary.csv` index all records and are the recommended starting point.
- Verification: `review/` contains the independent Warshall check, the Lean axiom report, and the record of the last full regression run.

C.2 Sections and records

Section 3 uses the cycle-proxy comparison and the counterexample search. Section 4 uses the reduction 2-complex records. Section 6 and Sections 7.3 and 7.4 use the string, term, and critical-pair records. Sections 7.1 and 7.2 use the enumeration and counterexample records and the independent check. Section 8 uses the completion and reduction 2-complex records. Earlier planning maps from sections and from table and figure slots to records are kept in `paper/notes/`; they predate some of the present tables and figures. The figures of this paper are drawn directly in its LaTeX sources from the examples they depict.

D Version History

Version 1 (17 March 2026). First release, DOI [10.5281/zenodo.19061345](https://doi.org/10.5281/zenodo.19061345).

Version 2 (1 October 2026). DOI [10.5281/zenodo.23088991](https://doi.org/10.5281/zenodo.23088991).

- The Lean development now covers arbitrary rewriting relations without finiteness assumptions, and adds termination, Newman’s lemma, the equivalence of confluence and elementary flatness under termination, and the equivalence of confluence and unique normal forms under normalization (Section 5).
- The negative result on graph flatness is stated for the two cycle-based proxies; the reduction graph of a finite system does determine its confluence (Section 3).
- Elementary holonomy is defined by joinability alone, and 2-cells are attached only to filled branchings (Section 4).
- Critical pairs are presented as generators of the nontrivial branchings rather than of all peaks, and the computations use concrete reachable critical-pair instances, including instances in context and under substitution (Section 6).
- The computational pipeline was corrected: self-overlaps of term rules are included, unification performs a full occurs check, substitution is simultaneous, invalid rules are rejected, and the critical-pair and holonomy analyses refuse to certify verdicts from incomplete explorations (Appendix B). All curated counts and verdicts are unchanged.
- New and redrawn figures, a table of smallest counterexamples, and a rewritten exposition.

References

- [1] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [2] John C. Baez and Javier P. Muniain. *Gauge Fields, Knots and Gravity*, volume 4 of *Series on Knots and Everything*. World Scientific Publishing Company, River Edge, NJ, 1994.
- [3] Nachum Dershowitz and Jean-Pierre Jouannaud. Rewrite systems. In Jan van Leeuwen, editor, *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics*, pages 243–320. Elsevier, 1990.

- [4] Yves Guiraud and Philippe Malbos. Higher-dimensional normalisation strategies for acyclicity. *Advances in Mathematics*, 231(3–4):2294–2351, 2012.
- [5] Yves Guiraud and Philippe Malbos. Polygraphs of finite derivation type. *Mathematical Structures in Computer Science*, 28(2):155–201, 2018.
- [6] Gérard P. Huet. Confluent reductions: Abstract properties and applications to term rewriting systems. *Journal of the ACM*, 27(4):797–821, 1980.
- [7] Donald E. Knuth and Peter B. Bendix. Simple word problems in universal algebras. In John Leech, editor, *Computational Problems in Abstract Algebra*, pages 263–297. Pergamon, Oxford, 1970.
- [8] Claudio Meneses. Thin homotopy and the holonomy approach to gauge theories. In Fernando Galaz-García, Cecilia González-Tokman, and Juan Carlos Pardo Millán, editors, *Mexican Mathematicians in the World: Trends and Recent Contributions*, volume 775 of *Contemporary Mathematics*, pages 233–253. American Mathematical Society, 2021.
- [9] Samuel Mimram. Categorical coherence from term rewriting systems. In Marco Gaboardi and Femke van Raamsdonk, editors, *8th International Conference on Formal Structures for Computation and Deduction (FSCD 2023)*, volume 260 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:17, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [10] Craig C. Squier. Word problems and a homological finiteness condition for monoids. *Journal of Pure and Applied Algebra*, 49(1–2):201–217, 1987.
- [11] Craig C. Squier and Friedrich Otto. The word problem for finitely presented monoids and finite canonical rewriting systems. In Pierre Lescanne, editor, *Rewriting Techniques and Applications*, volume 256 of *Lecture Notes in Computer Science*, pages 74–82, Berlin, Heidelberg, 1987. Springer.
- [12] Terese. *Term Rewriting Systems*. Number 55 in Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 2003.
- [13] Ioannis Tsiokos. Six birds: Foundations of emergence calculus, 2026. Manuscript. DOI: <https://doi.org/10.48550/arXiv.2602.00134>.
- [14] Ioannis Tsiokos. To count a stone with six birds: A mathematics is a theory, 2026. Manuscript. DOI: <https://doi.org/10.5281/zenodo.18402004>.
- [15] Ioannis Tsiokos. To lay a stone with six birds: Finite-state semantics for packaging, directionality, and coarse-graining, 2026. Manuscript. DOI: <https://doi.org/10.20944/preprints202602.1699.v1>.